

Lambda calcolo

Dispensa per il corso di Paradigmi di Programmazione
Laurea in Informatica, Università di Pisa

(Versione aggiornata al 25 settembre 2024)

Indice

1	Introduzione	2
1.1	Prerequisiti	2
1.2	Le origini nella teoria della calcolabilità	2
1.3	La nascita dei paradigmi imperativo e funzionale	5
2	Il λ-calcolo	6
2.1	Sintassi	6
2.2	Esecuzione: valutare le λ -espressioni	11
3	Confluenza e non terminazione	17
4	λ-calcolo e programmazione funzionale	18
4.1	Funzioni Curryed e di ordine superiore	18
4.2	Ricorsione: il combinatore Y	20
4.3	Rappresentare dati e controllo del flusso di esecuzione come funzioni	22
5	Strategie per il passaggio dei parametri: call-by-name (lazy) e call-by-value (eager)	23
A	Tabelle riassuntive delle definizioni	28

1 Introduzione

In questa dispensa sarà presentato il λ -calcolo come formalismo alla base della programmazione funzionale. Si partirà dalle origini storiche del formalismo, per poi passare alla descrizione degli aspetti sintattici e semantici, fino a trattare il rapporto tra il λ -calcolo e i linguaggi funzionali. La trattazione in termini matematici sarà accompagnata da riferimenti ed esempi relativi a linguaggi di programmazione comunemente utilizzati, come JavaScript e C. In Appendice si trovano riepilogate le definizioni formali.

1.1 Prerequisiti

Questa dispensa è stata scritta presupponendo che il lettore abbia una conoscenza di base dei seguenti argomenti:

- definizione di linguaggi formali tramite grammatiche;
- definizione di relazioni tramite regole di inferenza;
- programmazione in un linguaggio C-like come C, C++, Java, JavaScript o simili.

1.2 Le origini nella teoria della calcolabilità

Per ripercorrere le origini del λ -calcolo bisogna tornare all'inizio del secolo scorso, periodo in cui gli esponenti più autorevoli nell'ambito della matematica e della logica iniziarono a interrogarsi sulla possibilità di formalizzare tutte le conoscenze matematiche all'interno di un unico sistema logico, che consentisse, ad esempio, di esprimere tutte le dimostrazioni di tutti i teoremi.

In particolare, nel 1928, David Hilbert (1862-1943) formulò il quesito noto come *Entscheidungsproblem* (in italiano, *problema della decisione*), come segue:

È possibile definire una procedura, eseguibile del tutto meccanicamente, in grado di stabilire, per qualunque formula esprimibile nel linguaggio formale della logica del primo ordine, se tale formula sia o meno un teorema? In altri termini, se può essere dedotta a partire da dati assiomi (che includano almeno l'Aritmetica di Peano) usando le regole della logica stessa.

Il problema posto da Hilbert trova una prima risposta negativa nei Teoremi di Incompletezza proposti nel 1930 da Kurt Gödel (1906-1978). In sostanza, tali teoremi implicano che in qualunque sistema formale sufficientemente espressivo da includere l'Aritmetica di Peano è possibile esprimere delle formule che sono vere, ma non sono dimostrabili a partire dagli assiomi che definiscono il sistema formale e usando solo le regole della logica del primo ordine.

Al di là degli aspetti legati alla completezza dei sistemi formali, il quesito formulato da Hilbert aprì la strada anche allo studio di un'altra importante problematica: come si formalizza la nozione di procedura effettiva, eseguibile

in modo meccanico? In altri termini, come si esprime un algoritmo in modo formale? Come si può formalizzare la nozione intuitiva di che cosa vuol dire *calcolabile*? Questa domanda ha portato a diverse risposte, tra le quali quelle di Alonzo Church (1903-1995) e di Alan M. Turing (1912-1954) che introdussero due modelli diversi per definire che cosa è calcolabile.

- Church introdusse il Lambda calculus, nell'articolo "An unsolvable problem of elementary number theory", *Bulletin of the American Mathematical Society*, May 1935.
- Turing introdusse le Macchine di Turing, nell'articolo "On computable numbers, with an application to the Entscheidungsproblem", *Proceedings of the London Mathematical Society*, May 1935.

Questi due formalismi (assieme al formalismo delle *funzioni ricorsive generali*, proposto da Gödel nel 1931) hanno fornito contributi fondamentali alla teoria della calcolabilità, e hanno dato origine a famiglie di linguaggi di programmazione che si sono poi evolute nei linguaggi odierni.

Il *lambda calculus* (o lambda calcolo, o λ -calcolo) è un sistema formale che consente di rappresentare procedure di calcolo in termini di composizioni e applicazioni di funzioni minimali. Il λ -calcolo è l'oggetto di questa dispensa e sarà descritto dettagliatamente nel seguito.

Le macchine di Turing (*Turing machines*) sono invece un formalismo in cui le procedure di calcolo sono rappresentabili come delle macchine idealizzate dotate di un nastro infinito su cui possono leggere e scrivere simboli, e in cui la logica della procedura da eseguire è espressa in termini di un automa a stati finiti: partendo da uno stato iniziale, la "macchina" legge il simbolo corrente nel nastro e, sulla base di questo e delle transizioni disponibili, si sposta in un altro stato ed eventualmente aggiorna il simbolo sul nastro (anche scorrendolo). Questo modo di elaborare simboli leggendo e scrivendo astrae il modo in cui comunemente si calcola utilizzando carta e penna. Partendo da una sequenza di simboli di input (ad esempio un'espressione da risolvere) si leggono e scrivono simboli sul foglio seguendo regole ben definite (ad esempio, le regole dell'aritmetica).

Un'importante caratteristica delle macchine di Turing è che il nastro su cui vengono "memorizzati" i simboli da elaborare può essere usato anche per memorizzare una codifica della funzione che si vuole calcolare: in altre parole, il codice del programma da eseguire. L'automata che guida l'esecuzione dovrà quindi essere in grado di scandire la porzione di nastro che contiene il programma, per poi andare a eseguire le operazioni corrispondenti sulla porzione di nastro che contiene i dati.

Turing ha dimostrato che qualunque macchina definita per calcolare una specifica funzione (senza memorizzarne il codice nel nastro, ma con un automa di controllo definito *ad hoc* per quella funzione) può essere codificata come un programma da memorizzare su nastro. Nasce così il concetto di *Macchina di Turing Universale*, ossia un'unica macchina di Turing capace di "simulare" l'esecuzione di qualunque macchina di Turing opportunamente codificata su na-

stro, e quindi capace di calcolare qualunque funzione che possa essere calcolata tramite questo formalismo.

I lavori di Turing, così come quelli di Church sul λ -calcolo, sono tra i fondamentali principali della teoria della calcolabilità. Essi hanno consentito di comprendere che non tutte le funzioni che la matematica consente di definire possono essere calcolate in modo meccanico. In altri termini, esistono funzioni per le quali non è possibile definire una macchina di Turing, o un'espressione del λ -calcolo, o un programma in un qualunque linguaggio di programmazione moderno, che ne calcoli il risultato. Questi risultati, assieme ai teoremi di incompletezza di Gödel, hanno portato alla conclusione che il problema della decisione inizialmente proposto da Hilbert non fosse risolvibile.

Esempi di *funzioni non calcolabili* sono descritti solitamente sfruttando la capacità di autoreferenzialità dei sistemi formali considerati. Ad esempio, è possibile definire una macchina di Turing che verifichi se una qualunque altra macchina di Turing codificata su nastro sia scritta correttamente (dal punto di vista sintattico), mentre *non* è possibile definire una macchina di Turing che verifichi se un'altra macchina di Turing codificata su nastro, una volta eseguita, termini sempre la propria esecuzione (questo è noto come il *problema della fermata* o *halting problem*). Analogamente, non è possibile definire un'espressione del λ -calcolo che, date altre due espressioni del λ -calcolo, ne verifichi l'equivalenza.

Questi risultati ottenuti con formalismi diversi offrono in realtà punti di vista diversi sullo stesso panorama. Lavori successivi che dimostrano le corrispondenze tra macchine di Turing, espressioni del λ -calcolo, funzioni ricorsive generali, ecc... hanno portato gli scienziati a formulare la seguente ipotesi nota come *Tesi di Church-Turing*:

Se una funzione è calcolabile (tramite un qualunque formalismo), allora esiste una macchina di Turing in grado di calcolarla.

Questa tesi (indimostrabile, ma universalmente accettata) implica che esiste una classe di funzioni calcolabili che coincide con la classe di funzioni per le quali si può definire una macchina di Turing. Inoltre, dalle corrispondenze dimostrate con gli altri formalismi, segue che anche il λ -calcolo, le funzioni ricorsive generali, ma anche i linguaggi di programmazione moderni possono calcolare esattamente la stessa classe di funzioni. Sono tutti cioè *Turing-completi*. Ne consegue che, qualunque sia il linguaggio di programmazione utilizzato, non è possibile scrivere un programma che controlli se un altro programma termina sempre, e non è possibile scrivere un programma che controlli se altri due programmi sono equivalenti. Queste sarebbero funzionalità che ci piacerebbe poter avere nel nostro ambiente di sviluppo preferito, per controllare che il programma che stiamo scrivendo non si pianti all'infinito per colpa di un *bug* oppure per verificare che una nuova versione del programma continui a funzionare come la precedente. Tuttavia la teoria della calcolabilità ci dice che queste funzionalità non saranno mai disponibili.

1.3 La nascita dei paradigmi imperativo e funzionale

Il modello di calcolo delle macchine di Turing ha ispirato lo scienziato di origini ungheresi John von Neumann (1903-1957) nella definizione di un'architettura di elaborazione che è alla base del funzionamento degli attuali computer.

L'*architettura di von Neumann* prevede due componenti principali:

- la **memoria**, dove sono memorizzati sia i programmi da eseguire sia i dati da elaborare tramite tali programmi;
- l'**unità centrale di elaborazione (CPU)**, che ha il compito di eseguire i programmi immagazzinati in memoria prelevando le istruzioni (e i dati relativi), interpretandole ed eseguendole una dopo l'altra, aggiornando di conseguenza la memoria.

Le analogie concettuali tra l'architettura di von Neumann e la macchina di Turing universale sono chiare: la memoria corrisponde al nastro (che contiene anche il codice da eseguire) e la CPU corrisponde all'automa che controlla la macchina. Sebbene i computer moderni aggiungano a questa architettura tutta una serie di funzionalità ulteriori (ad esempio, l'esecuzione in pipeline, gli aspetti di parallelismo, ecc...) questo è tutt'ora un modello di riferimento valido.

Il fatto che le macchine di Turing imitino, in sostanza, il modo con cui si calcola usando carta e penna, che l'architettura di von Neumann sia ispirata alle macchine di Turing, e che i computer moderni si basino sull'architettura di von Neumann, ci porta a concludere che con i calcolatori di oggi quello che possiamo calcolare è in linea di principio ciò che saremmo in grado di calcolare usando carta e penna. Infine, la tesi di Church-Turing ci porta a concludere che tanto con carta e penna, quanto con un PC, la classe delle funzioni delle quali possiamo calcolare un risultato è la stessa.

Il fatto che già i primi calcolatori elettronici negli anni '50 si basassero sull'architettura di von Neumann (e quindi sui concetti definiti con le macchine di Turing) ha condizionato la definizione dei primi linguaggi di programmazione. I linguaggi in stile Turing-von Neumann sono caratterizzati dalla presenza di:

- **variabili**, che astraggono le locazioni di memoria
- **istruzioni di controllo**, come `test-and-jump`, `goto`, `if`, `while...`, che modificano il flusso di esecuzione dei programmi
- **istruzioni di assegnamento**, che modificano lo stato della macchina.

I linguaggi in questo stile sono detti *imperativi*, in quanto prevedono sequenze di *comandi* da impartire alla macchina. Tra i primi linguaggi di questo tipo possiamo menzionare certamente il Fortran, che è stato il primo vero linguaggio di programmazione di successo. Ancora oggi, grazie alla sua semplicità e all'efficienza nell'esecuzione, viene talvolta impiegato per implementare librerie matematiche o per il calcolo ad alte prestazioni. In seguito, grande fortuna hanno avuto i linguaggi Pascal e C, da cui, per evoluzioni successive, sono nati linguaggi quali C++, Java, C#, JavaScript, Python, ecc...

Parallelamente, il λ -calcolo ha influenzato la nascita di un'altra classe di linguaggi di programmazione non basati su comandi, ma strutturati principalmente come composizioni di funzioni. Questi linguaggi, chiamati *funzionali*, permettono di scrivere i programmi in modo simile a come si definiscono le funzioni matematiche. Esempi significativi di linguaggi funzionali delle prime generazioni sono LISP ed ML, più recentemente evolutisi in linguaggi come Haskell, OCaml, Scheme, Rust, ecc...

2 Il λ -calcolo

Descriviamo ora il λ -calcolo, formalismo, come abbiamo visto, alla base dei linguaggi di programmazione funzionali.

2.1 Sintassi

La sintassi del λ -calcolo è minimale: consente di scrivere espressioni che contengono solo dichiarazioni di funzioni e applicazioni di funzioni. In aggiunta, il linguaggio consente di introdurre variabili che sono utilizzate solitamente come parametri formali delle funzioni, ma non solo. Di seguito riportiamo la definizione formale.

Sintassi. Supponendo di avere un insieme infinito di possibili variabili $V = x, y, z, \dots$ la sintassi del λ -calcolo consiste di espressioni definite dalla seguente grammatica:

$$\begin{array}{ll} e ::= x & \text{(variabile)} \\ \quad | \lambda x.e & \text{(astrazione funzionale)} \\ \quad | e e & \text{(applicazione)} \end{array}$$

nella quale x rappresenta in realtà una qualunque variabile in V .

Il costrutto di astrazione funzionale $\lambda x.e$ dichiara una funzione anonima con parametro formale x e corpo e . Ad esempio, l'espressione

$$\lambda x.x$$

rappresenta la funzione che associa x a x , cioè la funzione *identità*, con x come parametro e sempre x come corpo.

Il costrutto di astrazione funzionale corrisponde ai costrutti per la definizione di funzioni anonime che si trovano in molti linguaggi di programmazione. Ad esempio, in Javascript la funzione identità può essere definita come funzione anonima nel modo seguente:

```
function(x) {return x;}
```

oppure, più brevemente, come segue:

```
x => x
```

Una funzione anonima del λ -calcolo (detta anche λ -astrazione) può poi essere applicata a un argomento attraverso il costrutto di applicazione e e, ossia semplicemente affiancando (giustapponendo) la definizione della funzione stessa (la prima espressione) alla definizione dell'argomento (la seconda espressione). Ad esempio, possiamo applicare la funzione identità a una nuova variabile y come segue:

$$(\lambda x.x)y$$

Le parentesi in questo esempio sono importanti per far comprendere che la variabile y è al di fuori della definizione della funzione. Omettendo le parentesi (in questo modo: $\lambda x.xy$) si starebbe invece definendo una funzione con un parametro x e con l'applicazione di x a y all'interno del suo corpo. Approfondiremo tra poco l'uso delle parentesi e le convenzioni sintattiche nelle λ -espressioni.

Vediamo alcuni esempi di λ -espressioni sintatticamente corrette:

$$(\lambda x.x)(\lambda y.y) \tag{1}$$

$$((\lambda x.x)(\lambda y.y))z \tag{2}$$

$$((\lambda x.(\lambda y.(xy)))z)k \tag{3}$$

$$((\lambda x.(\lambda y.(xy)))(\lambda z.z))k \tag{4}$$

In questi esempi abbiamo volutamente inserito le parentesi in modo esauritivo. Ricostruendo l'albero degli annidamenti di parentesi si otterrebbe una struttura (detta *albero di sintassi astratta*) che sostanzialmente ricalca l'albero di derivazione dell'espressione che consente di verificare che essa appartiene al linguaggio definito dalla grammatica.

Convenzioni sintattiche. Per evitare di appesantire le espressioni del λ -calcolo con così tante parentesi, si assume che:

1. l'astrazione sia associativa a destra, ovvero che lo *scope* (ambito di visibilità) del costruttore λ si estenda il più a destra possibile.

Ad esempio,

$$\lambda x.\lambda y.xy \text{ corrisponde a } \lambda x.(\lambda y.(xy))$$

e non a $\lambda x.((\lambda y.x)y)$, in cui lo scope di λy non arriverebbe a includere la y , e neanche a $(\lambda x.(\lambda y.x))y$, in cui lo scope di λx non arriverebbe a includere la y .

2. l'applicazione sia associativa a sinistra.

Ad esempio,

$$xyz \text{ corrisponde a } (xy)z$$

ossia prima si applica x a y , e poi si applica il risultato ottenuto a z . Analogamente,

$$(\lambda x.x)(\lambda y.y)z \text{ corrisponde a } ((\lambda x.x)(\lambda y.y))z$$

Alla luce di queste convenzioni, i quattro esempi di λ -espressioni illustrati sopra si possono riscrivere come segue:

$$(\lambda x.x)(\lambda y.y) \quad (5)$$

$$(\lambda x.x)(\lambda y.y)z \quad (6)$$

$$(\lambda x.\lambda y.xy)zk \quad (7)$$

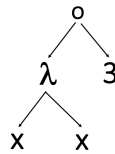
$$(\lambda x.\lambda y.xy)(\lambda z.z)k \quad (8)$$

Anche se non abbiamo ancora visto come si “eseguano” (o più propriamente si *valutano*) le λ -espressioni, possiamo descrivere i quattro esempi appena illustrati in termini di definizione e applicazione di funzioni. Ad esempio, nell'espressione (5) applichiamo la funzione identità (già vista) a un'altra definizione della stessa funzione, in cui cambia solo il nome del parametro formale. Nell'espressione (6) applichiamo la funzione identità a sé stessa, e il risultato (che sarà ancora la funzione identità) viene poi applicato a z . Nell'espressione (7) applichiamo una funzione che riceve un primo parametro x e successivamente un secondo parametro y , agli argomenti z e k . All'interno del suo corpo, la funzione applica x a y , quindi applicherà z a k . Infine, l'espressione (8) è simile a (7), ma il primo argomento passato è la funzione identità, che verrà applicata a k , come indicato dal corpo della funzione.

Alberi per rappresentare lambda espressioni. Può essere utile rappresentare una lambda espressione come albero. Questa rappresentazione si rivela utile per capire meglio la struttura delle lambda espressioni.

- L'albero per una variabile x è una foglia con etichetta x .
- Un'astrazione $\lambda x.e$ si rappresenta con un nodo λ che ha due sottoalberi come figli: il sottoalbero sinistro è dato dalla foglia x , mentre il sottoalbero destro è dato dall'albero che rappresenta e .
- L'applicazione di funzione $e_1 e_2$ è rappresentato con un nodo $\circ$ con gli alberi relativi a e_1 ed e_2 come figli.

Ad esempio, vediamo di seguito l'albero relativo all'applicazione $(\lambda x.x)3$



La struttura ad albero risulta utile per capire quali passi di valutazione sono appropriati.

Con questa ulteriore rappresentazione delle lambda espressioni si conclude la parte di spiegazione intuitiva, in cui abbiamo introdotto una serie di concetti che

vedremo di seguito in maggior dettaglio. Ad esempio, il fatto di poter definire funzioni con più di un parametro formale, oppure la possibilità di usare una funzione come argomento da passare a un'altra funzione.

Ruolo delle variabili. Prima di poter discutere degli aspetti più avanzati, è necessario riflettere sul ruolo delle variabili nelle λ -espressioni. Negli esempi che abbiamo considerato possiamo vedere che in alcuni casi sostituire una variabile con un'altra non è un problema. Ad esempio, la funzione identità la possiamo definire come $\lambda x.x$, ma anche come $\lambda y.y$. In entrambi i casi stiamo denotando la stessa entità, ovvero una funzione che ha come corpo il suo parametro, quindi al momento dell'applicazione restituirà esattamente l'argomento che le viene passato. In altre parole, sia $(\lambda x.x)z$ che $(\lambda y.y)z$ dovranno dare come risultato z . Se invece consideriamo le espressioni $(\lambda x.x)z$ e $(\lambda x.x)k$, in cui andiamo ad applicare la stessa funzione a due argomenti che rappresentano variabili distinte, il risultato che otterremo nei due casi sarà diverso: z nel primo e k nel secondo.

Il motivo per cui in $(\lambda x.x)z$ la variabile x può essere sostituita da y , mentre la variabile z non può essere sostituita da k è che x è *legata* dal costruttore λx , mentre z non lo è (si dice in questo caso che è *libera*, ovvero non legata da nessun λ). La differenza diventa evidente se pensiamo che il costruttore λx in un'astrazione funzionale definisce il parametro formale della funzione. Come in tutti i linguaggi di programmazione più comuni, i nomi assegnati ai parametri formali delle funzioni sono ininfluenti e possono sempre essere modificati. Le variabili libere, invece, come z nell'esempio, corrispondono a variabili che non sono usate come parametri formali di nessuna funzione. Tornando al paragone con i linguaggi di programmazione, le variabili libere le possiamo immaginare come variabili globali, oppure come variabili dichiarate in un blocco più esterno rispetto a quello corrente. Modificare queste variabili potrebbe portare a fare riferimento a una variabile globale diversa, alterando così il significato e l'esecuzione del programma.

È bene quindi essere sempre in grado di identificare le variabili libere all'interno di una λ -espressione. A tal fine, diamo la definizione di *insieme delle variabili libere* (Free Variables) FV di una λ -espressione e .

Insieme delle variabili libere. L'insieme delle variabili libere di una λ -espressione e , denotato con $FV(e)$, è definito ricorsivamente sulla struttura di e nel modo seguente:

$$\begin{aligned} FV(x) &= \{x\} \\ FV(\lambda x.e) &= FV(e) \setminus \{x\} \\ FV(e_1 e_2) &= FV(e_1) \cup FV(e_2) \end{aligned}$$

In sostanza, il calcolo di $FV(e)$ scende ricorsivamente nell'espressione accumulando (tramite l'unione) le variabili che incontra nelle sott'espressioni. Nel momento in cui, risalendo, si incontra una λ -astrazione, la variabile legata dal λ viene rimossa dall'insieme delle variabili libere calcolato nel corpo della funzione che essa definisce.

Riprendendo alcune espressioni già viste:

- $FV(\lambda x.x) = \emptyset$, poiché $FV(\lambda x.x) = FV(x) \setminus \{x\} = \{x\} \setminus \{x\} = \emptyset$;
- $FV((\lambda x.\lambda y.xy)(\lambda z.z)k) = \{k\}$, poiché

$$\begin{aligned}
& FV((\lambda x.\lambda y.xy)(\lambda z.z)k) \\
&= FV((\lambda x.\lambda y.xy)(\lambda z.z)) \cup FV(k) && \text{(ricordare associatività)} \\
&= (FV(\lambda x.\lambda y.xy) \cup FV(\lambda z.z)) \cup FV(k) \\
&= (FV(\lambda x.\lambda y.xy) \cup \emptyset) \cup \{k\} && \text{(saltati passi visti prima)} \\
&= (FV(\lambda y.xy) \setminus \{x\}) \cup \{k\} \\
&= ((FV(xy) \setminus \{y\}) \setminus \{x\}) \cup \{k\} \\
&= (((FV(x) \cup FV(y)) \setminus \{y\}) \setminus \{x\}) \cup \{k\} \\
&= (((\{x\} \cup \{y\}) \setminus \{y\}) \setminus \{x\}) \cup \{k\} \\
&= \{k\}
\end{aligned}$$

Inoltre, è interessante considerare i seguenti esempi (il cui calcolo è lasciato al lettore):

- $FV(\lambda x.xz) = \{z\}$
- $FV((\lambda x.xk)x) = \{k, x\}$

Il primo esempio mostra che una variabile libera può occorrere anche nel corpo di una definizione di funzione (che non abbia quella variabile come parametro formale, ovviamente). Il secondo invece mostra che una variabile legata non interferisce con eventuali variabili libere omonime che si trovino al di fuori del suo scope. Nell'esempio, la x in $\lambda x.xk$ e la x che si trova alla fine dell'espressione sono due variabili diverse: la prima è legata, mentre la seconda è libera (e viene raccolta in FV). Questo è esattamente ciò che accade anche nella maggior parte dei linguaggi di programmazione: in ambiti diversi del programma si può usare lo stesso nome per due variabili distinte.

Per concludere la trattazione della sintassi del λ -calcolo, introduciamo ora il concetto che espressioni che differiscono solo nel nome delle variabili legate e che sono da considerarsi equivalenti (le espressioni sono dette in questo caso *α -equivalenti*). Questo ci consentirà, in alcuni casi in cui potrebbero insorgere ambiguità sulle variabili, di rinominare le variabili legate ottenendo un'espressione equivalente a quella originale. Questa operazione di ridenominazione delle variabili legate è detta *α -conversione*. L'intuizione dietro questa nozione è che la scelta di una variabile legata, in un'astrazione, non ha importanza. Ad esempio, $\lambda x.x$ e $\lambda y.y$ sono termini α -equivalenti e rappresentano entrambi la stessa funzione, cioè la funzione identità. L'unica cosa che conta è il legame che inducono.

Nozioni di α -equivalenza e α -conversione. Due espressioni e_1 ed e_2 sono *α -equivalenti* (e si denota $e_1 \equiv_\alpha e_2$) se l'una può essere ottenuta dall'altra, rinominando una o più variabili legate, ovvero quando differiscono solo nei nomi delle variabili legate. Questo senza interferire con le variabili libere, ossia

senza che alcuna occorrenza di una variabile libera diventi legata a seguito della ridenominazione. Data un'espressione e_1 , l'operazione di ridenominazione delle variabili legate che consente di trasformarla in un'espressione e_2 tale che $e_1 \equiv_\alpha e_2$ è detta α -conversione.

La nozione di α -equivalenza, qui introdotta basandosi sul concetto intuitivo di “ridenominazione delle variabili legate”, sarà maggiormente chiara più avanti, dopo aver introdotto l'operazione più generale di “capture avoiding substitution”.

Facciamo qualche esempio di espressioni α -equivalenti ottenibili tramite α -conversioni in cui le istanze legate di x e y vengono ridenominate:

- $\lambda x.\lambda y.xyz \equiv_\alpha \lambda y.\lambda x.yxz$
- $(\lambda x.x)x \equiv_\alpha (\lambda y.y)x$

Vediamo inoltre qualche interessante esempio di espressioni che non sono invece α -equivalenti (e scriveremo $\not\equiv_\alpha$ per indicarlo)

1. $\lambda x.\lambda y.xyz \not\equiv_\alpha \lambda y.\lambda x.xyz$
2. $\lambda x.xy \not\equiv_\alpha \lambda y.yy$
3. $(\lambda x.xy)y \not\equiv_\alpha (\lambda y.yy)y$

Nel primo esempio, le due espressioni non sono equivalenti perché le occorrenze di x e y nel corpo della λ -astrazione non sono state sostituite. Scambiare i nomi di due variabili in due costruttori λ richiede che anche tutte le occorrenze delle variabili a essi legati siano rinominate di conseguenza. Intuitivamente è sicuro rinominare una variabile se si cambiano in modo consistente anche tutti i riferimenti a quella variabile.

Il secondo esempio è invece più interessante. Quello che accade è che a seguito della ridenominazione della variabile legata x in y , l'occorrenza di y nel corpo della λ astrazione venga “catturata” dal nuovo λy , trasformando cioè y , prima libera, in una variabile legata. Infatti, $FV(\lambda x.xy) = \{y\}$, mentre $FV(\lambda y.yy) = \emptyset$.

Il terzo esempio risente esattamente dello stesso problema, sebbene l'insieme delle variabili libere sia uguale nelle due espressioni (ed è $\{y\}$). Quello che cambia è che comunque la prima occorrenza di y viene catturata, quindi nell'espressione a sinistra entrambe le occorrenze di y sono libere, mentre nell'espressione a destra solo l'ultima occorrenza di y è tale.

Le α -conversioni che portano a questo tipo di interferenze tra le variabili libere e quelle legate non sono ammissibili.

2.2 Esecuzione: valutare le λ -espressioni

Poiché il λ -calcolo è un linguaggio basato su espressioni, per descrivere i processi che portano al calcolo di un risultato, è più appropriato parlare di “valutazione” piuttosto che di “esecuzione”. La *valutazione delle λ -espressioni* è un processo

che consiste in una serie di passi di calcolo che possono portare a un risultato finale. Dal momento che il λ -calcolo prevede di fatto solo funzioni (anonime) e un costrutto per la loro applicazione, i passi di calcolo da svolgere durante la valutazione saranno in sostanza tutti passi di applicazione di funzione.

Se pensiamo alle espressioni aritmetiche, un passo di calcolo corrisponde a:

1. individuare un'operazione all'interno dell'espressione che è pronta per essere calcolata (ad esempio perché i suoi operandi non contengono ulteriori operazioni);
2. sostituire tale operazione con il suo risultato.

Ad esempio, all'interno della seguente espressione aritmetica:

$$3 + \underline{(5 - 2)} + 3 - 1$$

possiamo individuare l'operazione $5 - 2$ (sottolineata per chiarezza) come “pronta per essere calcolata” e la possiamo sostituire con il suo risultato ottenendo

$$3 + 3 + 3 - 1$$

La scelta dell'operazione da elaborare può non essere unica (potevamo anche scegliere $3 - 1$) e il procedimento va ripetuto finché non si ottiene un'espressione non ulteriormente elaborabile (ad esempio, un singolo numero intero). Quello sarà il risultato della valutazione dell'espressione.

Nel λ -calcolo il procedimento è analogo, solo che l'unica operazione di calcolo che possiamo eseguire è l'applicazione funzionale. Si dovranno eseguire gli stessi due passi che abbiamo visto sopra: individuare l'operazione da eseguire e sostituirla con il suo risultato.

L'operazione di applicazione funzionale del λ -calcolo prende il nome di β -riduzione, e la sottoespressione (porzione) di λ -espressione in cui si esegue tale operazione e che viene sostituita dal suo risultato prende il nome di *redex* o espressione riducibile (*reducible expression*). Vediamo un esempio con la funzione identità:

$$\underline{(\lambda x.x)y}$$

In questo esempio stiamo applicando la funzione identità alla variabile libera y . Abbiamo quindi una funzione e un argomento: l'intera λ -espressione è un redex pronto per essere sostituito con il risultato dell'applicazione.

L'applicazione funziona come segue: prende l'argomento e lo sostituisce sintatticamente a tutte le occorrenze del parametro formale che si trovano nel corpo della funzione. Ciò che si ottiene dopo dalla sostituzione sarà il risultato dell'applicazione, o *ridotto*.

Nell'esempio della funzione identità, il corpo della funzione è x , che è un'occorrenza del parametro formale. Sostituendolo sintatticamente all'argomento (che è y) otteniamo come risultato y . Quindi $(\lambda x.x)y$ dopo un passo dà come risultato y , e la valutazione si ferma in quanto y non è un redex (non contiene

funzioni pronte per essere applicate). Si dice quindi che il risultato della valutazione della λ -espressione è y , e questo è anche intuitivamente corretto: la funzione identità ha restituito esattamente l'argomento che le era stato passato.

D'ora in avanti descriveremo un passo di β -riduzione tramite una freccia $\rightarrow$, quindi il nostro esempio può essere formalmente rappresentato come segue:

$$(\lambda x.x)y \rightarrow y$$

Vediamo alcuni altri esempi, che prevedono anche più di un passo di calcolo e in cui per chiarezza a ogni passo sottolineiamo il redex che viene elaborato. Partiamo da questo:

$$(\lambda x.x)(\lambda y.y)z \rightarrow (\lambda y.y)z \rightarrow z$$

Qui stiamo applicando la funzione identità alla funzione identità, e il risultato dovrà essere applicato alla variabile libera z . Il risultato del primo passo (in cui si sostituisce $\lambda y.y$ a x nel corpo della prima funzione) è la funzione identità applicata a z . Il secondo passo, quindi, dà come risultato z .

In questo esempio è bene notare che, date le convenzioni sintattiche che prevedono l'associatività a sinistra dell'applicazione funzionale, le funzioni vanno applicate esattamente in quell'ordine. Sarebbe sbagliato applicare prima $(\lambda y.y)$ a z (anche se il risultato in questo caso sarebbe lo stesso) e questo diventa chiaro se esplicitiamo le parentesi che sono sottointese per via dell'associatività: $((\lambda x.x)(\lambda y.y))z$.

Consideriamo ora i seguenti due esempi:

$$(\lambda x.\lambda y.xy)(\lambda z.z)k \rightarrow (\lambda y.((\lambda z.z)y))k \rightarrow (\lambda y.y)k \rightarrow k \quad (9)$$

$$(\lambda x.\lambda y.xy)(\lambda z.z)k \rightarrow (\lambda y.((\lambda z.z)y))k \rightarrow (\lambda z.z)k \rightarrow k \quad (10)$$

In entrambi, la λ -espressione da valutare è la stessa, ma durante l'esecuzione vengono fatti passi diversi in quanto (al secondo passo) ci sono due redex possibili tra i quali scegliere.

Nel primo caso viene scelto il redex più interno, $(\lambda z.z)y$, mentre nel secondo il redex più esterno, $(\lambda y.((\lambda z.z)y))k$. In entrambi i casi abbiamo una λ -astrazione (nel secondo caso un po' più complicata) applicata a un argomento. Il risultato che otteniamo alla fine è tuttavia lo stesso: k .

Visti questi esempi possiamo definire un redex come una struttura $(\lambda x.e_1)e_2$ che deve essere individuata nell'espressione per poter applicare la β -riduzione. Nell'individuare il redex bisogna assicurarsi che non ci siano parentesi aperte o chiuse (anche implicite) tra la λ -astrazione $(\lambda x.e_1)$ e l'argomento e_2 . È possibile che nell'espressione questa struttura ricorra più volte (anche l'una dentro l'altra) e questo comporta la scelta tra diversi modi di valutare l'espressione.

Una volta individuato il redex, è necessario poi prestare attenzione alla sostituzione sintattica che viene effettuata con la β -riduzione, poiché potrebbe causare problemi con le variabili. Se le espressioni coinvolte nella sostituzione infatti condividono nomi di variabili, il significato dell'espressione risultante potrebbe cambiare se la sostituzione non ne tenesse conto. Ad esempio in

$$(\lambda x.k)x \quad (11)$$

se andassimo a sostituire l'argomento x al parametro formale x come abbiamo fatto negli esempi precedenti, otterremmo $\lambda x.x$. Purtroppo, però, questo cambierebbe il significato dell'espressione, perché questa applicazione della sostituzione trasformerebbe una funzione costante che restituisce sempre k nella funzione identità. L'occorrenza di x passata come argomento alla funzione era una variabile libera nell'espressione iniziale e non è la stessa x che è legata come argomento alla λ . Dopo la sostituzione, invece, la x inserita nel corpo della funzione viene catturata dal costruttore λx rendendola legata!

Analoga questione si porrebbe se andassimo a sostituire l'argomento y al parametro formale x nell'espressione $(\lambda x.\lambda y.xy)y$, ottenendo $\lambda y.yy$, dove la y inserita nel corpo della funzione al costruttore verrebbe catturata da λy .

Questo modo di eseguire la sostituzione non è quindi corretto, e va modificato in modo da tenere conto delle variabili libere presenti nell'argomento passato alla funzione e da gestire l'eventuale conflitto tra nomi di variabili. Per questo va introdotta la nozione di *capture avoiding substitution*, ovvero il processo che evita che le variabili libere della sostituzione vengano accidentalmente catturate all'interno dell'espressione originale.

Capture avoiding substitution. Date due espressioni e_1 ed e_2 , e una variabile x , definiamo la *capture avoiding substitution* di x in e_1 con e_2 , denotata $e_1\{x := e_2\}$ ricorsivamente sulla struttura di e_1 come segue:

$$\begin{aligned}
x\{x := e_2\} &= e_2 \\
y\{x := e_2\} &= y \quad \text{se } x \neq y \\
(e'e'')\{x := e_2\} &= (e'\{x := e_2\})(e''\{x := e_2\}) \\
(\lambda x.e)\{x := e_2\} &= \lambda x.e \\
(\lambda y.e)\{x := e_2\} &= \lambda y.(e\{x := e_2\}) \quad \text{se } x \neq y \text{ e } y \notin FV(e_2) \\
(\lambda y.e)\{x := e_2\} &= \lambda z.((e\{y := z\})\{x := e_2\}) \quad \text{se } x \neq y, y \in FV(e_2) \\
&\quad \text{e } z \text{ fresca}
\end{aligned}$$

I primi due casi della definizione dicono che se applichiamo la sostituzione $\{x := e_2\}$ a un'espressione costituita solo da una variabile, la sostituzione avrà luogo solo se tale variabile è proprio x . Questo è il caso base della ricorsione sulla struttura della λ -espressione. Se invece (terzo caso) l'espressione a cui applichiamo la sostituzione è un'applicazione $e'e''$, allora la sostituzione dovrà semplicemente propagarsi ricorsivamente sulle due sottoespressioni.

I casi interessanti sono gli ultimi, quelli relativi alla λ -astrazione. Qui possiamo trovarci in tre situazioni distinte. La prima (più semplice) è quella in cui x , parametro formale della λ -astrazione, coincide con la variabile da sostituire. In questo caso la ricorsione si arresta e la sostituzione non viene applicata, perché le sole occorrenze di x che si possono sostituire sono quelle libere. La seconda è quella in cui y , parametro formale della λ -astrazione, non coincide

con la variabile da sostituire e non appartiene alle variabili libere di e_2 . In questo caso si può procedere con la sostituzione nel corpo della λ -astrazione. Se invece y , oltre a essere il parametro formale, fosse anche una variabile libera di e_2 (terzo caso), rischieremmo di trovarci nella situazione descritta prima, in cui una variabile libera (di e_2 , in questo caso) vien erroneamente catturata da un costruttore λ , diventando così legata. La soluzione per evitarlo, adottata in questo caso, è quella di fare una ulteriore sostituzione. Prima di applicare la sostituzione $\{x := e_2\}$ nel corpo della λ -astrazione, è necessario cioè rinominare il parametro formale y ricorrendo a una variabile z che non compaia da nessun'altra parte nella λ -espressione (altrimenti si potrebbero creare nuovi conflitti). Una variabile al momento non utilizzata è detta *fresca*. Va notato che la sostituzione va fatta anche nel costruttore λ , cioè λy diventa λz . In questo modo si ottiene un'espressione α -equivalente alla precedente in cui viene meno il conflitto di nomi tra il parametro formale e la variabile libera, e quindi la sostituzione può avvenire senza problemi.

Vediamo qualche esempio:

- $((\lambda x.yx)w)\{x := \lambda k.kx\} = (\lambda x.yx)w$
- $((\lambda x.yx)w)\{y := \lambda k.kx\} \equiv_\alpha ((\lambda z.yz)w)\{y := \lambda k.kx\} = \lambda z.(\lambda k.kx)z)w$
- $((\lambda x.yx)w)\{w := \lambda k.kx\} = (\lambda x.yx)(\lambda k.kx)$

Nel primo esempio la sostituzione non ha luogo in quanto x non è libera. Nel secondo esempio l'espressione $\lambda k.kx$ contiene la variabile libera x che rischia di essere catturata. Per questo motivo, prima di operare la sostituzione vera e propria è necessario rinominare il parametro formale x usando una variabile fresca z . Nel terzo caso, la sostituzione avviene invece senza problemi.

L'operazione di capture avoiding substitution ci consente ora di definire la β -riduzione in modo da evitare il problema con le variabili libere descritto in precedenza, e di valutare correttamente anche espressioni come (11).

β -riduzione La regola di β -riduzione descrive un passo di calcolo di una λ -espressione. Identificato un *redex* $(\lambda x.e_1)e_2$ all'interno dell'espressione, questo viene elaborato come indicato dalla seguente regola:

$$(\lambda x.e_1)e_2 \rightarrow e_1\{x := e_2\}$$

il risultato ottenuto sostituisce il redex, mentre tutto il resto dell'espressione rimane inalterato.

Quindi, riprendendo la λ -espressione in (11), la sua valutazione sarà la seguente:

$$(\lambda x.\lambda y.xy)y \equiv_\alpha (\lambda x.\lambda z.xz)\underline{y} \rightarrow \lambda z.yz$$

In questo esempio, il primo passo di α -conversione è esplicitato per mostrare l'intervento della capture avoiding substitution nell'introdurre la variabile fresca z per evitare di catturare y . Come in precedenza, il redex a cui si applica la riduzione (in questo caso l'intera espressione) è sottolineato per chiarezza.

La valutazione completa di una λ -espressione è quindi una sequenza di passi di applicazione della β -riduzione che porta a un'espressione non ulteriormente riducibile. Tali espressioni sono dette in *forma normale β* , e rappresentano di fatto i *valori* che si possono ottenere come risultato. Non potendo contenere redex (altrimenti sarebbero ulteriormente riducibili) queste particolari espressioni/valori possono sostanzialmente assumere due forme: un'astrazione funzionale, oppure un'espressione che contiene variabili libere che “ostacolano” la riduzione ulteriore. Alcuni esempi di valori di questo tipo sono i seguenti.

- $\lambda x.x$
- $\lambda x.xy$
- x
- xy

I primi due casi rappresentano funzioni. Quindi possiamo dire che *nel λ -calcolo le funzioni sono valori*, e questo è forse la principale caratteristica del paradigma di programmazione funzionale, come si riscontra in molti linguaggi moderni, e di cui il λ -calcolo rappresenta il fondamento teorico. Gli ultimi due casi invece rappresentano valori che, per via delle variabili libere, sono intuitivamente non del tutto definitivi. Se sapessimo come istanziare tali variabili, potremmo in certi casi continuare con il calcolo. Ma questo è puramente un ragionamento teorico. All'atto pratico la valutazione dell'espressione si arresta raggiungendo quei risultati.

La β -riduzione fornisce le basi per poter definire una nozione di equivalenza tra λ -espressioni, detta *β -equivalenza*, che tiene conto della valutazione delle espressioni, e non solo della loro sintassi.

β -equivalenza. Due λ -espressioni e_1 ed e_2 si dicono *β -equivalenti*, indicato $e_1 \equiv_\beta e_2$, se vale una delle seguenti condizioni:

- $e_1 \equiv_\alpha e_2$
- $e_1 \Rightarrow e'_1 \equiv_\alpha e_2$ oppure $e_2 \Rightarrow e'_2 \equiv_\alpha e_1$
- $e_1 \Rightarrow e'_1$ e $e_2 \Rightarrow e'_2$ con $e'_1 \equiv_\alpha e'_2$

dove $\Rightarrow$ rappresenta una sequenza di β -riduzioni consecutive (formalmente, $\Rightarrow$ è la chiusura transitiva della relazione $\rightarrow$).

Due espressioni sono *β -equivalenti* se (e solo se) porteranno allo stesso risultato. Questo accade (i) se le due espressioni sono uguali a meno di rinominare le variabili legate (cioè sono α -equivalenti), (ii) se una si riduce all'altra (di nuovo, a meno di α -equivalenza), o (iii) se entrambe si riducono a una stessa espressione (sempre modulo α -equivalenza). Quindi, ad esempio, valgono le seguenti

equivalenze:

$$\begin{aligned} (\lambda x.x) &\equiv_{\beta} (\lambda y.y) \\ (\lambda x.\lambda y.xy)(\lambda x.x)z &\equiv_{\beta} (\lambda k.k)z \\ (\lambda x.\lambda y.xy)(\lambda x.x)(\lambda x.x)z &\equiv_{\beta} (\lambda x.x)((\lambda x.\lambda y.x)zk) \end{aligned}$$

Si lascia al lettore verificare quale caso della β -equivalenza si applichi, e che la valutazione delle λ -espressioni equivalenti porti allo stesso risultato.

3 Confluenza e non terminazione

Abbiamo visto che la β -riduzione consente di valutare le λ -espressioni ottenendo un risultato. Abbiamo anche osservato che, durante la valutazione, è possibile trovarsi di fronte a più redex pronti per essere elaborati, il che richiede di decidere quale scegliere. Questo accadeva negli esempi in (9) e (10) visti in precedenza. In quegli esempi, entrambe le scelte operate al secondo passo conducevano in ogni caso allo stesso risultato finale k .

Il fatto che comunque si scelga di elaborare una λ -espressione si ottenga sempre lo stesso risultato non è scontato. Esistono formalismi (o linguaggi) in cui questa proprietà non vale.

Pensiamo ad esempio alla valutazione delle espressioni Booleane in molti linguaggi di programmazione. Ad esempio, in linguaggi C-like (come C, C++, Java, JavaScript, ecc..), si può scrivere un `if` simile al seguente:

```
if (den!=0 && num/den>0) {
    ...
}
```

Se gli operandi del `&&` potessero essere valutati in ordine arbitrario (come accade nella logica con l'operazione di congiunzione), a seconda che si scelga di valutare prima `den!=0` o `num/den>0` si potrebbero ottenere due risultati diversi quando `den` è zero: rispettivamente `false` e un errore di esecuzione.

Altri esempi si possono trovare facilmente nella programmazione concorrente: quando due thread concorrenti operano sulla stessa area di memoria, i valori finali nella memoria condivisa possono variare a seconda di quale thread venga eseguito per primo.

Nel λ -calcolo fortunatamente questi problemi non si verificano. Vale infatti una proprietà, detta di *confluenza* delle computazioni che è sancita dal Teorema di Church-Rosser¹.

Teorema di Church-Rosser. Data una λ -espressione e , se esistono due sequenze di β -riduzioni diverse che consentono di ridurre e a e_1 o a e_2 non equivalenti, allora esiste anche una λ -espressione e' tale che entrambe e_1 ed e_2 potranno essere ridotte a e' (o a qualcosa di α -equivalente).

¹J. Barkley Rosser (1907–1989) era uno studente di Church.

Il teorema implica che qualunque scelta si operi nell'applicare la β -riduzione, le espressioni che si ottengono potranno poi sempre portare verso lo stesso risultato. L'ordine in cui vengono scelte le riduzioni non fa quindi differenza. Questa proprietà prende il nome di *confluenza* proprio per richiamare il fatto che i diversi flussi di esecuzione della valutazione vanno a riunirsi (a confluire) nello stesso risultato finale. Una conseguenza del teorema è che se un termine possiede una forma normale β , essa è unica, a meno di ridenominazioni.

È importante sottolineare che il teorema dice che e_1 ed e_2 *possono* essere entrambi ridotti a e' , ossia, esistono delle sequenze di β -riduzioni che trasformano entrambe le espressioni in e' . La domanda che ci si può porre ora è: *la valutazione di una λ -espressione potrebbe non terminare?*

La risposta alla domanda è affermativa: è possibile definire λ -espressioni non terminanti. Un esempio classico è la λ -espressione nota come *combinatore* Ω , definito come segue:

$$\Omega = (\lambda x.xx)(\lambda x.xx)$$

È facile accorgersi che applicando la β -riduzione a Ω si ottiene nuovamente Ω , e questo porta ad una computazione infinita:

$$\Omega = (\lambda x.xx)(\lambda x.xx) \rightarrow (\lambda x.xx)(\lambda x.xx) \rightarrow \dots$$

l'equivalente di un ciclo infinito in un linguaggio di programmazione.

Inoltre, può succedere che a seconda delle scelte che si fanno quando ci sono più redex disponibili, la valutazione della stessa λ -espressione possa terminare o meno, come in questi casi, in cui il redex scelto ad ogni passo è sottolineato:

- $(\lambda x.k)\underline{\Omega} \rightarrow k$
- $(\lambda x.k)\underline{\Omega} \rightarrow (\lambda x.k)\underline{\Omega} \rightarrow k$
- $(\lambda x.k)\underline{\Omega} \rightarrow (\lambda x.k)\underline{\Omega} \rightarrow \dots (\lambda x.k)\underline{\Omega} \rightarrow \dots$

Quello che ci dice il Teorema di Church-Rosser è che, anche in questi casi, tutte le computazioni *che terminano* portano allo stesso risultato di valutazione.

4 λ -calcolo e programmazione funzionale

Vediamo ora perché un formalismo così minimale come il λ -calcolo è considerato il fondamento teorico della programmazione funzionale.

4.1 Funzioni Curryed e di ordine superiore

Come già detto in precedenza, uno degli elementi chiave della programmazione funzionale è il fatto che *le funzioni sono valori*, e come tali possono essere il risultato di una computazione, ma possono anche essere passate come argomenti ad altre funzioni. Quest'ultimo aspetto consente di definire *funzioni di ordine superiore* (o *higher-order*), ossia funzioni che ammettono funzioni come

argomento o risultato. Un esempio banale è la funzione *Apply*, definita come segue:

$$Apply = (\lambda f. \lambda x. f x)$$

La funzione *Apply* prende una funzione *f*, un argomento *x* e poi applica *f* ad *x*. Quindi, assumendo di dare il nome *Id* alla funzione identità, definita come $Id = (\lambda x. x)$, avremo:

$$Apply\ Id\ k \rightarrow (\lambda x. Id\ x)k \rightarrow Id\ k \rightarrow k$$

Si noti che *Apply* è apparentemente una funzione con due parametri, mentre il λ -calcolo prevede astrazioni funzionali con un solo parametro. Il modo per definire una funzione che lavora su più parametri consiste nell'annidare le astrazioni funzionali (scrivendo cioè $\lambda x. \lambda y. \dots$), trasformando in questo caso una funzione con due argomenti in una catena di due funzioni ognuna con un solo argomento.

Le funzioni con più parametri definite nel modo del λ -calcolo vengono dette *funzioni Curried*, dal nome del matematico e logico Haskell B. Curry (1900-1982) che aveva introdotto questa tecnica. Esse sono un altro aspetto caratteristico della programmazione funzionale e hanno la particolarità di poter essere applicate parzialmente. Ad esempio, se proviamo a passare solo il primo argomento alla funzione *Apply*:

$$Apply\ Id$$

ciò che otteniamo con un passo di β -riduzione tenendo conto di come è definita la funzione *Apply*, è:

$$\lambda x. Id\ x$$

ossia una funzione con un parametro formale *x* a cui sarà applicata la funzione *Id*. Applicando la funzione *Apply* parzialmente (passandole cioè solo il primo argomento) otteniamo come risultato una funzione pronta a ricevere il secondo argomento per completare il lavoro.

Per fare un esempio più concreto, supponiamo per un attimo di avere anche una operazione di somma + e di poter usare valori interi nelle λ -espressioni. La funzione che fa la somma di due numeri potrebbe quindi essere definita come segue:

$$Somma = (\lambda x. \lambda y. x + y)$$

tale che

$$Somma\ 10\ 5 \rightarrow \lambda y. 10 + y \rightarrow 10 + 5 = 15$$

Applicando parzialmente la funzione si otterrebbe:

$$Somma\ 10 \rightarrow \lambda y. 10 + y$$

cioè una funzione pronta a sommare 10 al valore che le viene passato.

A questo punto, continuando a definire funzioni, potremmo immaginarci di dare un nome a tale funzione ottenuta come risultato:

$$SommaDieci = Somma\ 10$$

potendo a questo punto scrivere:

$$SommaDieci\ 5 = 15 \quad SommaDieci\ 3 = 13 \quad SommaDieci\ 10 = 20 \quad ecc...$$

4.2 Ricorsione: il combinatore Y

Nel λ -calcolo non è previsto un meccanismo nativo (ad es., un costrutto sintattico specifico) per definire funzioni ricorsive. Le funzioni sono anonime, quindi non è possibile definire funzioni ricorsive richiamando la funzione stessa nel suo corpo² e non esistono altri operatori che consentano di iterare o eseguire ricorsivamente una data funzione. Ciò nonostante, un meccanismo di ricorsione può essere *implementato/codificato* usando i pochi costrutti sintattici che il formalismo mette a disposizione.

Una funzione può essere resa ricorsiva passandola ad un'altra particolare funzione definita negli anni '40 da Curry, e che prende il nome di Y *combinator* (o combinatore Y), definito come segue:

$$Y = \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$$

Notare che come per tutti i combinatori si tratta di un termine chiuso (privo cioè di variabili libere): $FV(Y) = \emptyset$. Data una qualunque λ -espressione F (che in pratica dovrà definire una funzione affinché la cosa abbia senso) il combinatore Y soddisfa la seguente proprietà di *punto fisso*:

$$YF \equiv_{\beta} F(YF)$$

cioè Y applicato a una funzione F si riduce all'applicazione di F a YF . Y permette dunque di applicare una funzione un numero arbitrario di volte. Questa proprietà può essere verificata mostrando che sia YF che $F(YF)$ si riducono dopo pochi passi di β -riduzione alla stessa λ -espressione, come segue:

$$\begin{aligned} YF &= \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))F \\ &\rightarrow (\lambda x.F(xx))(\lambda x.F(xx)) \\ &\rightarrow F((\lambda x.F(xx))(\lambda x.F(xx))) \end{aligned}$$

$$\begin{aligned} F(YF) &= F(\lambda f.(\lambda x.f(xx))(\lambda x.f(xx))F) \\ &\rightarrow F((\lambda x.F(xx))(\lambda x.F(xx))) \end{aligned}$$

da cui, per il terzo caso della definizione di β -equivalenza, si può concludere che $YF \equiv_{\beta} F(YF)$.

Si tratta pertanto di un operatore che restituisce un punto fisso per qualsiasi funzione di ingresso F gli venga passato.

Ora, come si utilizza il combinatore Y per definire funzioni ricorsive? Si può notare che quando utilizzato come costruttore di funzioni, crea un punto fisso sull'argomento che, quando applicato, si richiama. In questo modo, una funzione ricorsiva può essere ottenuta passando la stessa funzione come argomento e

²Anche negli esempi precedenti in cui abbiamo dato nomi di convenienza alle funzioni *Apply*, *Id*, *Somma*, ecc... non stavamo effettivamente dichiarando funzioni. Non è quindi possibile, nel λ -calcolo, scrivere qualcosa tipo $R = \lambda x.R$ in quanto $=$ non fa parte della sintassi del linguaggio.

utilizzando quell'argomento per effettuare la chiamata ricorsiva, anziché usare direttamente il nome della funzione. Più in dettaglio il modo è il seguente:

- si definisce una λ -espressione e al cui interno si usa un nome f per effettuare le chiamate ricorsive. Di fatto, e rappresenta il corpo della funzione ricorsiva f che si intende definire;
- si definisce la λ -astrazione $G = \lambda f.e$
- si definisce $F = YG$

. Così facendo, F sarà la funzione ricorsiva che si voleva definire.

Ad esempio, consideriamo l'espressione

$$e = \lambda y.(fy)y$$

che rappresenta una funzione che applica “ricorsivamente” f al suo parametro y , aggiungendo una istanza di y in fondo. Definiamo la λ -astrazione:

$$G = \lambda f.e$$

e applichiamo il combinatore Y :

$$F = YG$$

Applicando F a un argomento k , la sua esecuzione sarà la seguente:

$$\begin{aligned} & YGk \\ \Rightarrow & G(\lambda x.G(xx))(\lambda x.G(xx))k \\ \rightarrow_{\equiv_{\beta}} & (\lambda y.((YG)y)y)k \\ \rightarrow & (YG)kk \\ \Rightarrow & (YG)kkk \\ \rightarrow & \dots \end{aligned}$$

dove $\Rightarrow$, come prima, rappresenta la chiusura transitiva di $\rightarrow$, quindi una sequenza di passi di β -riduzioni non illustrati in dettaglio per semplicità. Inoltre, la β -equivalenza $\equiv_{\beta}$ nello sviluppo è usata per poter indicare più semplicemente (YG) nell'espressione anziché l'espressione più complessa (ma equivalente) che in realtà si otterrebbe.

Un esempio più concreto lo si può osservare in un linguaggio di programmazione che preveda funzioni anonime in stile λ -calcolo. Vediamo come si può implementare il combinatore Y in JavaScript e utilizzarlo per implementare una funzione per il calcolo del fattoriale usando solo funzioni anonime³:

```
const Y = f => (x => x(x))(x => f(y => x(x)(y)))
const factorial = f => (x => (x===1 ? 1 : x * f(x-1)))
const ris = Y(factorial)(10)
```

³Il combinatore Y implementato in questo esempio è leggermente diverso da quello definito sopra come λ -espressione per evitare che l'interprete JavaScript vada in stack overflow.

4.3 Rappresentare dati e controllo del flusso di esecuzione come funzioni

Brevemente, illustriamo come altri costrutti comuni nei linguaggi di programmazione funzionali possano essere codificati in λ -calcolo. Questo ci consentirà di considerare il λ -calcolo come un buon modello di questa famiglia di linguaggi.

Letterali Booleani. Le costanti *True* e *False* possono essere codificate dalle seguenti λ -astrazioni:

$$True = \lambda t. \lambda f. t \qquad False = \lambda t. \lambda f. f$$

ossia come funzioni con due parametri formali che restituiscono rispettivamente il primo o il secondo parametro.

Ci sono inoltre λ -termini che rappresentano le operazioni sui booleani. Ad esempio:

$$Not = \lambda x. x FT$$

funziona come un *not*:

$$Not\ T = (\lambda x. x\ FT)T \rightarrow TFT \rightarrow (\lambda t. \lambda f. t)FT \Rightarrow F$$

$$Not\ F = (\lambda x. x\ FT)F \rightarrow FFT \rightarrow (\lambda t. \lambda f. f)FT \Rightarrow T$$

Letterali numerici (naturali). Il numero naturale n è codificato come la λ -espressione C_n definita come segue:

$$C_n = \lambda z. \lambda s. s(s(\dots(s\ z))\dots)$$

dove s è ripetuto n volte. L'idea è che il parametro z rappresenti lo zero e il parametro s la funzione successore. Quindi $C_0 = \lambda z. \lambda s. z$, $C_1 = \lambda z. \lambda s. s(z)$, $C_2 = \lambda z. \lambda s. s(s(z))$, ecc...

Espressioni condizionali. Tenendo conto della codifica dei Booleani data prima, un costrutto di *espressione condizionale*⁴ può essere definito come segue:

$$IF = \lambda c. \lambda then. \lambda else. c\ then\ else$$

quindi, ad esempio, l'espressione $IF\ True\ e_1\ e_2$ avrà il seguente sviluppo:

$$\begin{aligned} IF\ True\ e_1\ e_2 &= (\lambda c. \lambda then. \lambda else. c\ then\ else)\ True\ e_1\ e_2 \\ &\rightarrow (\lambda then. \lambda else. True\ then\ else)\ e_1\ e_2 \\ &\rightarrow (\lambda else. True\ e_1\ else)\ e_2 \\ &\rightarrow True\ e_1\ e_2 \\ &= (\lambda t. \lambda f. t)\ e_1\ e_2 \\ &\rightarrow (\lambda f. e_1)\ e_2 \\ &\rightarrow e_1 \end{aligned}$$

⁴Da non confondere con il *comando* di scelta condizionale **if** tipico dei linguaggi imperativi

Operazioni su naturali. Tenendo conto della rappresentazione dei numeri naturali vista sopra, è possibile definire le operazioni aritmetiche come λ -astrazioni che prendono come parametri gli operandi n ed m . Queste le definizioni, ad esempio, delle operazioni di somma e moltiplicazione:

$$\begin{aligned} Plus &= \lambda m. \lambda n. \lambda z. \lambda s. (m (n z s) s) \\ Times &= \lambda m. \lambda n. (m C_0 (Plus n)) \end{aligned}$$

Anche se non è semplice, lasciamo al lettore verificare l'esecuzione di operazioni quali:

$$Plus C_3 C_2 \quad \text{e} \quad Times C_3 C_2$$

Oltre a quanto visto in questa sezione, è possibile codificare in λ -calcolo anche costrutti per la dichiarazione di variabili locali, strutture dati (coppie, liste, ...), ecc. Il concetto importante da comprendere è che nella sua semplicità il λ -calcolo è un valido modello dei linguaggi di programmazione funzionale in quanto consente di codificarne tutti i principali costrutti linguistici.

5 Strategie per il passaggio dei parametri: call-by-name (lazy) e call-by-value (eager)

Nelle sezioni precedenti abbiamo visto che ci possono essere più modi per valutare una λ -espressione, e che il Teorema di Church-Rosser garantisce che tutte le riduzioni che terminano porteranno allo stesso valore. Tuttavia quando ci si trova nella situazione di dover scegliere tra due o più redex pronti per essere ridotti, su quali criteri conviene basare la propria scelta? In altre parole, è possibile definire una *strategia di valutazione* che risolva le scelte e renda il processo di valutazione deterministico?

Chiaramente si possono definire diverse strategie di valutazione, ma è interessante considerarne due in particolare che trovano riscontri nei linguaggi di programmazione attuali: la strategia *call-by-name* (che porta a una valutazione *lazy*) e la strategia *call-by-value* (che porta a una valutazione *eager*). Entrambe riguardano nello specifico il passaggio dei parametri.

Consideriamo questo esempio, in cui va fatta attenzione all'uso delle parentesi:

$$(\lambda x. xx)((\lambda y. yyy)k)$$

i redex tra cui scegliere per applicare la β -riduzione sono due:

$$\underline{(\lambda x. xx)((\lambda y. yyy)k)} \quad \text{e} \quad (\lambda x. xx)(\underline{(\lambda y. yyy)k})$$

È facile notare che entrambe le scelte portano in due passi allo stesso risultato, ovvero $(kkk)(kkk)$.

In questo esempio ci troviamo in pratica di fronte a una funzione pronta per essere applicata a un argomento che potrebbe a sua volta essere immediatamente elaborato. Dobbiamo quindi scegliere se applicare prima la funzione più esterna

(passandole l'argomento così com'è definito e posticipando la valutazione di quest'ultimo), oppure se prima valutare l'argomento e poi applicare la funzione più esterna al risultato ottenuto.

La prima scelta è dettata dalla strategia *call-by-name*, in cui l'argomento viene passato così com'è scritto, e corrisponde alla valutazione *lazy* (pigra, poichè si posticipa la valutazione dell'argomento). La seconda scelta è invece dovuta alla strategia *call-by-value*, in cui la funzione viene applicata al valore ottenuto dalla valutazione dell'argomento e corrisponde alla valutazione *eager* (avida, impaziente) che valuta subito l'argomento.

La scelta di quando valutare gli argomenti passati a una funzione è critica anche nei linguaggi di programmazione. La maggior parte dei linguaggi utilizza un approccio *eager*. Ad esempio, usando la sintassi di C (e derivati), la chiamata di funzione

`f(3+4);`

porta a calcolare prima la somma, e poi a passare il risultato 7 alla funzione.

Questa scelta comporta delle conseguenze. Ad esempio, se la chiamata fosse la seguente

`f(g(0)+1);`

avremmo che viene prima eseguita la funzione *g* e poi la funzione *f*. Se entrambe le funzioni, ad esempio, lavorassero su una stessa variabile globale, potremmo avere un'interferenza di *g* sul calcolo che dovrà effettuare *f* di cui dobbiamo essere consapevoli.

Esistono linguaggi che prevedono la valutazione *lazy* (l'esempio classico è il linguaggio Haskell, nome scelto in onore dell'omonimo matematico), oppure che prevedono operatori per specificare che una certa espressione debba essere valutata in modo *lazy* (come OCaml). La valutazione *lazy*, se ben implementata, offre vantaggi prestazionali (si valuta l'argomento solo se necessario) e di espressività (ad esempio, per lavorare con strutture dati infinite). Allo stesso tempo, potrebbe portare a valutare più volte lo stesso argomento, in caso di più occorrenze. Tuttavia, la valutazione *eager* è nella stragrande maggioranza dei casi preferita in quanto molto più semplice per il programmatore.

Studiamo però le differenze tra le due strategie nel λ -calcolo. Per fare questo è opportuno formalizzare la β -riduzione tramite regole di inferenza. Questo ci consentirà di definire formalmente le strategie.

Per cominciare, la relazione di β -riduzione standard (senza strategie) può essere formalizzata come segue:

$$\begin{array}{c}
 (\lambda x.e_1)e_2 \rightarrow e_1\{x := e_2\} \\
 \\
 \frac{e_1 \rightarrow e'_1}{e_1 e_2 \rightarrow e'_1 e_2} \quad \frac{e_2 \rightarrow e'_2}{e_1 e_2 \rightarrow e_1 e'_2} \quad \frac{e \rightarrow e'}{\lambda x.e \rightarrow \lambda x.e'}
 \end{array}$$

L'assioma di questa definizione con regole di inferenza è quello già visto come definizione della β -riduzione: nel corpo dell'astrazione funzionale $(\lambda x.e_1)$ si

sostituisce ogni occorrenza libera di x con l'argomento e_2 . Le tre regole di inferenza, invece, esprimono formalmente il fatto che la riscrittura descritta dall'assioma si possa applicare a una qualunque sotto-espressione della λ -espressione che stiamo processando, cosa che in precedenza era stata specificata nel testo della definizione di β -riduzione, ma che ora appare formalizzata in termini matematici.

Le tre regole di inferenza, quindi, consentono di ridurre sotto-espressioni senza vincolare la scelta su quale di esse elaborare.

Riconsiderando l'esempio dell'espressione $(\lambda x.xx)((\lambda y.yyy)k)$ abbiamo infatti che la definizione ci consente di generare entrambi i seguenti alberi di derivazione, per il primo passo di β -riduzione:

$$\overline{(\lambda x.xx)((\lambda y.yyy)k)} \rightarrow \overline{((\lambda y.yyy)k)((\lambda y.yyy)k)} \quad (12)$$

e

$$\frac{(\lambda y.yyy)k \rightarrow kkk}{(\lambda x.xx)((\lambda y.yyy)k) \rightarrow (\lambda x.xx)(kkk)} \quad (13)$$

Consideriamo ora la valutazione *lazy* (strategia *call-by-name*), che può essere definita come segue:

$$(\lambda x.e_1)e_2 \rightarrow e_1\{x := e_2\}$$

$$\frac{e_1 \rightarrow e'_1}{e_1 e_2 \rightarrow e'_1 e_2}$$

In questo caso, la precedenza nella scelta tra i diversi redex disponibili deve essere data all'applicazione della funzione che si trova a un livello sintattico più alto (cioè, più esterno). Questa strategia si ottiene rimuovendo dalla definizione le regole di inferenza che consentono di ridurre l'argomento di una applicazione funzionale e il corpo di una funzione. La strategia impone così di portare, usando l'unica regola di inferenza disponibile, l'espressione in una forma in cui $(\lambda x.e_1)e_2$ è a top-level, e poi di applicare la sostituzione di x con e_2 senza poter prima ridurre e_1 o e_2 . In questo modo, tornando all'esempio, l'albero di derivazione in (13) non è più valido, mentre rimane possibile effettuare la derivazione in (12).

Infine, consideriamo la valutazione *eager* (strategia *call-by-value*), che definiamo come segue:

$$(\lambda x.e_1)v \rightarrow e_1\{x := v\}$$

$$\frac{e_1 \rightarrow e'_1}{e_1 e_2 \rightarrow e'_1 e_2} \quad \frac{e_2 \rightarrow e'_2}{v e_2 \rightarrow v e'_2}$$

Qui la precedenza deve essere data alla riduzione dell'argomento della funzione, prima che all'applicazione. Per questo modifichiamo l'assioma della definizione utilizzando il simbolo v per esprimere una λ -espressione non ulteriormente riducibile (ossia un valore). In questo modo, in un'applicazione funzionale, la

funzione può essere applicata solo dopo aver ridotto completamente l'argomento. A questo punto, rispetto alla strategia call-by-name, reintroduciamo la regola di inferenza che consente di ridurre l'argomento di una applicazione funzionale, ma con un accorgimento: anziché consentire di ridurre liberamente l'argomento e_2 , consentiamo a questa regola di applicarsi solo dopo che la funzione da applicare è stata completamente ridotta, ossia quando abbiamo ve_2 . In questo modo la strategia richiederà di ridurre la funzione da applicare (per portarla in forma $\lambda x.e_1$, ma senza scendere nel corpo e_1) per poi procedere con la riduzione dell'argomento e_2 prima di poter applicare.

Tornando ancora all'esempio, abbiamo che con questa strategia l'albero di derivazione in (12) non sarà più valido, mentre rimane possibile effettuare la derivazione in (13).

Attenzione! Nelle due strategie appena descritte, gioca un ruolo importante l'assenza della seguente regola di inferenza, che consente di eseguire parte del corpo di una funzione prima di applicarla ad un argomento:

$$\frac{e \rightarrow e'}{\lambda x.e \rightarrow \lambda x.e'}$$

senza questa regola abbiamo che, ad esempio, la seguente espressione non può essere ridotta:

$$\lambda x.((\lambda y.y)z)$$

quando invece usando la β -riduzione standard (senza strategie), tale espressione farebbe un passo di riduzione arrivando in $\lambda x.z$.

Questo significa che le strategie di call-by-name e call-by-value **non sono equivalenti** alla β -riduzione standard in quanto possono portare a risultati diversi ($\lambda x.((\lambda y.y)z)$ invece che $\lambda x.z$, nell'esempio). D'altronde, il modo di procedere delle strategie call-by-name e call-by-value è quello adottato anche in tutti i linguaggi di programmazione: il corpo di una funzione non viene mai eseguito (nemmeno parzialmente) prima dell'invocazione della funzione stessa. Ad esempio, se in C abbiamo la funzione:

```
int foo(int x) {
    int y = 5;
    int z = 6+y;
    return x+z;
}
```

sebbene si potrebbe calcolare il valore di **z**, prima ancora di chiamare la funzione, questo non viene fatto. Il corpo viene (completamente) eseguito solo nel momento in cui da qualche parte viene effettuata, ad esempio, la chiamata `foo(5)`.

È bene notare, a questo punto, che entrambe le strategie call-by-name e call-by-value sono *deterministiche*. Per qualunque λ -espressione abbiamo sempre una sola possibile regola applicabile. Questo fa sì che queste regole possano essere la base su cui definire un interprete (cioè un esecutore) di un linguaggio

di programmazione funzionale che, chiaramente, deve essere in grado di eseguire i programmi in modo ben determinato e non ambiguo.

L'ultima questione da chiarire ora è: le strategie call-by-name e call-by-value sono del tutto equivalenti? Il Teorema di Church-Rosser risponde in parte a questa domanda: sebbene le due strategie non siano equivalenti alla β -riduzione standard, non porteranno mai la stessa espressione a essere valutata con due risultati completamente diversi. Consideriamo però il seguente esempio:

$$IF\ True\ True\ (\Omega\ \Omega)$$

in cui IF , $True$ e Ω sono definiti come nelle sezioni precedenti. In questo caso è facile osservare che le due strategie portano a comportamenti diversi.

- Nel caso della *call-by-name* i tre argomenti dell' IF vengono passati così come sono, quindi IF riconosce immediatamente che la guardia è $True$ e restituisce il valore nel primo ramo: $True$.
- Nel caso della *call-by-value*, invece, i tre argomenti devono essere ridotti prima di essere passati all' IF . Tra questi, però, c'è $\Omega\ \Omega$ la cui valutazione non termina, portando l'intera espressione a non poter essere valutata.

Questo esempio illustra che le due strategie possono differire dal punto di vista della terminazione e, in particolare, la strategia call-by-value può in certi casi non terminare quando quella call-by-name potrebbe invece terminare. Ciò nonostante, come già detto, la maggior parte dei linguaggi di programmazione segue un approccio call-by-value (eager) per la sua maggiore semplicità di comprensione per il programmatore. Sarà compito del programmatore fare attenzione a non passare alle funzioni argomenti la cui valutazione potrebbe non terminare.

Definizioni per il λ -calcolo (non tipato)

28